VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

REPORT
ROS2

HW 3

Student's name: Vo Nguyen Bao Ngoc

Student ID: 2452840

Ho Chi Minh City, March 2026

Contents

1 Introduction **3**

2 Environment Setup **3**

3 Lab 1 – NAV2 System Observation **3**
3.1 Objective . . . 3
3.2 Commands Used . . . 3
3.3 Results . . . 3
3.4 Analysis . . . 5
3.5 Discussion . . . 5

4 Lab 2 – SLAM Mapping **5**
4.1 Objective . . . 5
4.2 Commands Used . . . 6
4.3 Results . . . 6
4.4 Map File Explanation . . . 6
4.5 Analysis . . . 7
4.6 Discussion . . . 7

5 Lab 3 – Costmap and Inflation Layer **7**
5.1 Objective . . . 7
5.2 Commands Used . . . 7
5.3 Parameter Configuration . . . 7
5.4 Results . . . 8
5.5 Analysis . . . 10
5.6 Discussion . . . 10

6 Lab 4 – Sending Navigation Goal using C++ **11**
6.1 Objective . . . 11
6.2 Implementation . . . 11
6.3 Code . . . 11
6.4 Execution . . . 14
6.5 Results . . . 14
6.6 Analysis . . . 15
6.7 Discussion . . . 15

7 Lab 5 – Planner and Controller Configuration **16**
7.1 Objective . . . 16
7.2 Configuration . . . 16
7.3 Execution . . . 16
7.4 Results . . . 17
7.5 Analysis . . . 18
7.6 Discussion . . . 18

8 Lab 6 – Behavior Tree in NAV2 **19**
8.1 Objective . . . 19
8.2 Overview . . . 19
8.3 Default Behavior Tree . . . 19
8.4 Analysis of Behavior Tree . . . 19
8.5 Modification . . . 20
8.6 Results . . . 20
8.7 Analysis . . . 20
8.8 Discussion . . . 21

9 Homework 1 – Patrol Robot **21**
9.1 Objective . . . 21
9.2 Requirements . . . 21
9.3 Implementation . . . 21
9.4 Code . . . 22
9.5 Execution . . . 25
9.6 Results . . . 25
9.7 Analysis . . . 26
9.8 Discussion . . . 26

10 Homework 2 – Planner Comparison using Metrics **27**
10.1 Objective . . . 27
10.2 Methodology . . . 27
10.3 Implementation . . . 27
10.4 Data Collection . . . 27
10.5 Results . . . 28
10.6 Analysis . . . 28
10.7 Discussion . . . 28

11 Homework 3 – Custom NAV2 Controller Plugin **28**
11.1 Objective . . . 28
11.2 Overview . . . 28
11.3 Implementation Details . . . 28
11.4 Parameters . . . 29
11.5 Plugin Registration . . . 29
11.6 Integration with NAV2 . . . 29
11.7 Results . . . 30
11.8 Analysis . . . 30
11.9 Discussion . . . 31

1 Introduction

This report presents the study and implementation of the ROS2 Navigation Stack (NAV2) using TurtleBot3 in a Gazebo simulation environment. The goal is to understand how different components in NAV2 interact, including nodes, topics, actions, and mapping using SLAM Toolbox.

2 Environment Setup

- Operating System: Ubuntu 22.04
- ROS2 Distribution: Humble
- Robot Platform: TurtleBot3 Burger
- Simulation Tool: Gazebo
- Programming Language: C++

3 Lab 1 – NAV2 System Observation

3.1 Objective

The objective of this lab is to observe and analyze the ROS2 Navigation (NAV2) system, including active nodes, topics, and action servers.

3.2 Commands Used

```
ros2 node list
ros2 topic list
ros2 topic info /scan
ros2 topic info /odom
ros2 topic info /cmd_vel
ros2 topic info /map
ros2 topic info /amcl_pose
ros2 action list
ros2 action info /navigate_to_pose
```

3.3 Results

Observed Nodes (partial):

- `/amcl`
- `/planner_server`
- `/controller_server`
- `/bt_navigator`
- `/map_server`

- `/behavior_server`
- `/waypoint_follower`
- `/global_costmap/global_costmap`
- `/local_costmap/local_costmap`

Observed Topics:

- `/scan` (sensor_msgs/msg/LaserScan): Publisher = 1, Subscriber = 4
- `/odom` (nav_msgs/msg/Odometry): Publisher = 1, Subscriber = 2
- `/cmd_vel` (geometry_msgs/msg/Twist): Publisher = 5, Subscriber = 1
- `/map` (nav_msgs/msg/OccupancyGrid): Publisher = 1, Subscriber = 3
- `/amcl_pose` (geometry_msgs/msg/PoseWithCovarianceStamped): Publisher = 1, Subscriber = 0

Observed Actions:

- `/navigate_to_pose`

Action Info:

- Action server: `/bt_navigator`
- Action clients:
  - `/rviz2`
  - `/waypoint_follower`
  - `/bt_navigator`
  - `/rviz_navigation_dialog_action_client`

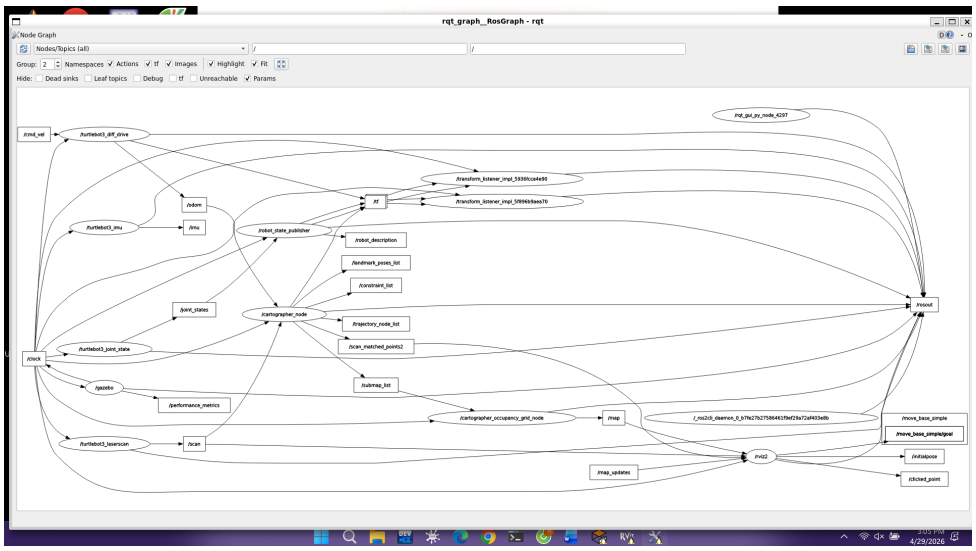

Figure 1: Node communication graph using `rqt_graph`

3.4 Analysis

- The topic `/cmd_vel` has multiple publishers (5) and one subscriber. This indicates that several components in NAV2 may generate velocity commands, while the robot base subscribes to execute motion.
- The topic `/scan` provides LiDAR data and is used by multiple components such as localization and costmap generation.
- The topic `/odom` provides raw odometry data, which is used by localization and navigation modules but may accumulate drift over time.
- The topic `/amcl_pose` has no subscribers, meaning it is mainly used for visualization (e.g., RViz) and monitoring the estimated pose.
- The action `/navigate_to_pose` is handled by the `/bt_navigator`, which acts as the main decision-making component in NAV2 using Behavior Trees.
- The NAV2 system is modular, with separate nodes for planning (`/planner_server`), control (`/controller_server`), and recovery behaviors (`/behavior_server`).

3.5 Discussion

1. Which node publishes `/cmd_vel`? Which node subscribes to it?

The topic `/cmd_vel` has multiple publishers, including components from the navigation stack such as the controller server and velocity smoother. The subscriber is the robot base (e.g., `/turtlebot3_diff_drive`), which executes the velocity commands to move the robot.

2. What is the difference between `/amcl_pose` and `/odom`?

The topic `/odom` provides raw odometry data based on wheel encoders and is subject to drift over time. In contrast, `/amcl_pose` provides a corrected estimate of the robot's position using the AMCL localization algorithm, which incorporates sensor data such as LiDAR and the map. Therefore, `/amcl_pose` is more accurate and stable than `/odom`.

3. Why are there two costmaps in NAV2?

NAV2 uses two costmaps for different purposes. The global costmap is used for long-range path planning across the entire map, while the local costmap is used for real-time obstacle avoidance and short-term navigation around the robot. This separation improves efficiency and allows the robot to react quickly to dynamic obstacles while still following a global plan.

4 Lab 2 – SLAM Mapping

4.1 Objective

The objective of this lab is to generate a map using SLAM Toolbox and understand how map data is stored and used in navigation.

4.2 Commands Used

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
ros2 launch slam_toolbox online_async_launch.py use_sim_time:=True
ros2 run teleop_twist_keyboard teleop_twist_keyboard

ros2 run nav2_map_server map_saver_cli -f ~/maps/turtlebot3_world
```

4.3 Results

Generated Map:

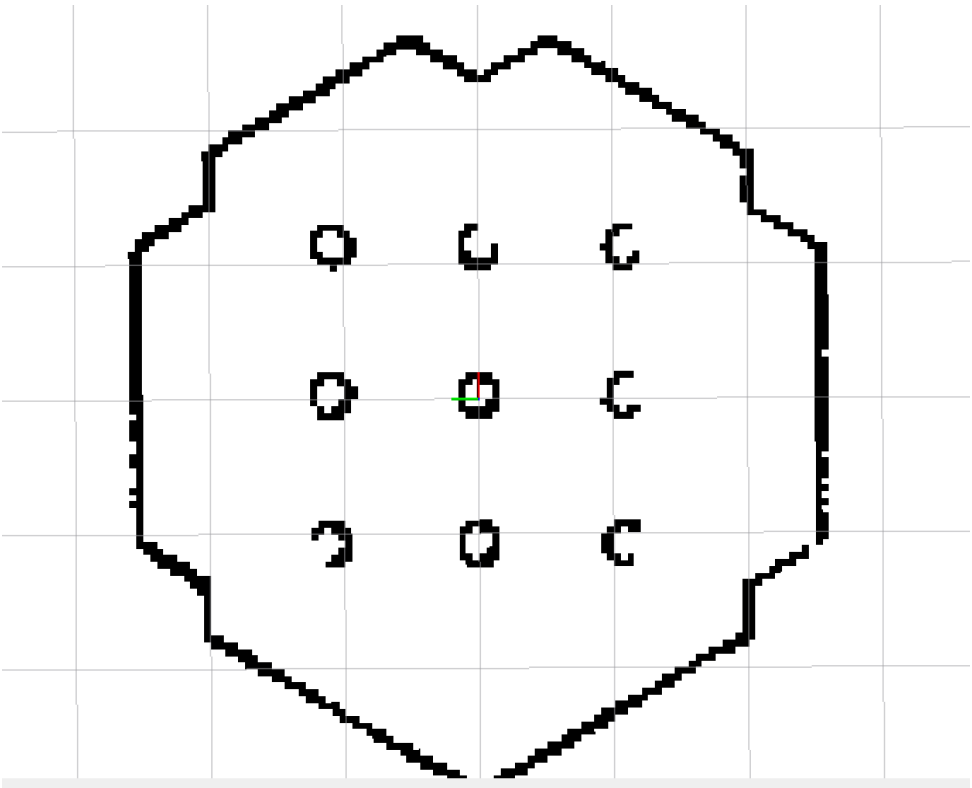

Figure 2: Occupancy grid map generated by SLAM Toolbox

Saved Files:

- `turtlebot3_world.pgm`
- `turtlebot3_world.yaml`

4.4 Map File Explanation

- **PGM file:**
  - 0: Occupied (obstacle)
  - 254: Free space
  - 205: Unknown area
- **YAML file:**

- `resolution`: Size of each pixel (meters/pixel)
- `origin`: Position of the map origin
- `occupied_thresh`: Threshold to classify obstacles
- `free_thresh`: Threshold to classify free space

4.5 Analysis

- A smaller resolution results in a more detailed map but increases memory usage and computation cost.
- If the robot moves too fast or rotates abruptly, the generated map may become distorted due to inaccurate sensor readings.
- Smooth and consistent robot motion leads to higher-quality mapping results.

4.6 Discussion

1. How does resolution affect the map?

The map resolution determines the size of each grid cell. A smaller resolution (e.g., 0.025 m/pixel) produces a more detailed and accurate map but requires more memory and computation. A larger resolution (e.g., 0.05 m/pixel) reduces computational cost but results in a less detailed map.

2. What happens if the robot moves too fast during mapping?

If the robot moves too fast or rotates abruptly during mapping, the SLAM algorithm may not correctly align sensor data. This can lead to distorted or inconsistent maps, such as overlapping walls or misaligned structures. For better mapping quality, the robot should move slowly and smoothly.

(Observation)

In our experiment, smoother and slower robot motion resulted in a more accurate and consistent map.

5 Lab 3 – Costmap and Inflation Layer

5.1 Objective

The objective of this lab is to understand how costmaps are constructed in NAV2 and how the inflation layer affects robot navigation behavior.

5.2 Commands Used

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py
use_sim_time:=True map:=$HOME/maps/turtlebot3_world.yaml
```

5.3 Parameter Configuration

The inflation radius parameter was modified in the costmap configuration file:

```
inflation_layer:
plugin: "nav2_costmap_2d::InflationLayer"
cost_scaling_factor: 3.0
inflation_radius: <value>
```

Three values were tested:

- 0.2 m
- 0.55 m
- 1.0 m

5.4 Results

Due to limitations in running Gazebo simulation, the following observations are based on the expected behavior of the NAV2 system.
Case 1: inflation radius = 0.2

- Robot behavior: The robot moves aggressively and follows paths very close to obstacles. It tends to take narrow passages with minimal clearance.
- Distance to obstacles: Close

.png

Figure 3: Costmap with inflation radius = 0.2

Case 2: inflation radius = 0.55

- Robot behavior: The robot moves smoothly and maintains a safe distance from obstacles. Navigation is stable and efficient.
- Distance to obstacles: Safe

.png

Figure 4: Costmap with inflation radius = 0.55

Case 3: inflation radius = 1.0

- Robot behavior: The robot becomes overly conservative and avoids areas near obstacles. In narrow environments, it may fail to find a valid path.
- Navigation success: May fail in narrow spaces

.png

Figure 5: Costmap with inflation radius = 1.0

5.5 Analysis

- When the inflation radius is small (0.2 m), the robot tends to move closer to obstacles. This increases the risk of collision but allows more freedom in narrow spaces.
- With a medium inflation radius (0.55 m), the robot maintains a safe distance from obstacles while still being able to navigate efficiently. This provides a good balance between safety and mobility.
- When the inflation radius is large (1.0 m), obstacles occupy a larger area in the costmap. This may prevent the robot from finding a valid path, especially in narrow environments.
- The parameter `cost_scaling_factor` affects how quickly the cost decreases as the distance from obstacles increases. A higher value results in a steeper gradient.

5.6 Discussion

1. How does cost_scaling_factor affect the inflation layer?

The `cost_scaling_factor` controls how quickly the cost values decay from obstacles. A higher value produces a steeper gradient, causing the robot to avoid obstacles more aggressively.

2. What happens if inflation_radius is too large?

If the inflation radius is larger than the available free space (e.g., corridor width), the robot may not find any valid path. This results in navigation failure.

6 Lab 4 – Sending Navigation Goal using C++

6.1 Objective

The objective of this lab is to implement a ROS2 C++ node that sends navigation goals to the NAV2 stack using an action client. The node communicates with the `/navigate_to_pose` action server and handles goal response, feedback, and result.

6.2 Implementation

A ROS2 package was created to implement the action client using `rclcpp_action`. The node is responsible for sending a goal position and monitoring the navigation process.

Key steps:

- Create an action client for `/navigate_to_pose`
- Wait for the action server to be available
- Define a goal using `geometry_msgs::msg::PoseStamped`
- Convert yaw angle to quaternion
- Send goal asynchronously
- Handle callbacks (goal response, feedback, result)

6.3 Code

Main implementation:

```
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "nav2_msgs/action/navigate_to_pose.hpp"
#include "geometry_msgs/msg/pose_stamped.hpp"
#include <cmath>

using NavigateToPose = nav2_msgs::action::NavigateToPose;
using GoalHandleNav  = rclcpp_action::ClientGoalHandle<NavigateToPose>;

class GoalSenderNode : public rclcpp::Node
{
public:
  GoalSenderNode() : Node("goal_sender")
  {
    // Tao action client ket noi toi /navigate_to_pose
    client_ = rclcpp_action::create_client<NavigateToPose>(this, "navigate_to_pose");
```

```
  }

  void send_goal(double x, double y, double yaw)
  {
    // Doi action server san sang (toi da 5 giay)
    if (!client_->wait_for_action_server(std::chrono::seconds(5))) {
      RCLCPP_ERROR(get_logger(), "Action server /navigate_to_pose khong san sang!");
      rclcpp::shutdown();
      return;
    }

    // Tao goal message
    auto goal_msg = NavigateToPose::Goal();
    goal_msg.pose.header.frame_id = "map";
    goal_msg.pose.header.stamp    = now();

    // Dat toa do dich
    goal_msg.pose.pose.position.x = x;
    goal_msg.pose.pose.position.y = y;
    goal_msg.pose.pose.position.z = 0.0;

    // Chuyen yaw (radian) sang quaternion
    // Cong thuc: z = sin(yaw/2), w = cos(yaw/2)
    goal_msg.pose.pose.orientation.x = 0.0;
    goal_msg.pose.pose.orientation.y = 0.0;
    goal_msg.pose.pose.orientation.z = std::sin(yaw / 2.0);
    goal_msg.pose.pose.orientation.w = std::cos(yaw / 2.0);

    // Gan 3 callback
    auto send_goal_options = rclcpp_action::Client<NavigateToPose>::SendGoalOptions()

    send_goal_options.goal_response_callback =
      std::bind(&GoalSenderNode::GoalResponseCallback, this, std::placeholders::_1);

    send_goal_options.feedback_callback =
      std::bind(&GoalSenderNode::FeedbackCallback, this,
        std::placeholders::_1, std::placeholders::_2);

    send_goal_options.result_callback =
      std::bind(&GoalSenderNode::ResultCallback, this, std::placeholders::_1);

    // Gui goal
    client_->async_send_goal(goal_msg, send_goal_options);
    RCLCPP_INFO(get_logger(), "Da gui goal: x=%.2f  y=%.2f  yaw=%.2f rad", x, y, yaw)
  }

private:
  rclcpp_action::Client<NavigateToPose>::SharedPtr client_;
```

```
  //  Callback 1: Server chap nhan hay tu choi goal
  void GoalResponseCallback(const GoalHandleNav::SharedPtr & goal_handle)
  {
    if (!goal_handle) {
      RCLCPP_ERROR(get_logger(), "Goal bi TU CHOI boi action server!");
    } else {
      RCLCPP_INFO(get_logger(), "Goal duoc CHAP NHAN. Robot bat dau di chuyen...");
    }
  }

  //  Callback 2: Feedback dinh ky tu server
  void FeedbackCallback(
    GoalHandleNav::SharedPtr,
    const std::shared_ptr<const NavigateToPose::Feedback> feedback)
  {
    auto & pos = feedback->current_pose.pose.position;
    double eta = feedback->estimated_time_remaining.sec
               + feedback->estimated_time_remaining.nanosec * 1e-9;

    RCLCPP_INFO(get_logger(),
      "[Feedback] Vi tri hien tai: (%.2f, %.2f) | ETA: %.1f s | So lan recovery: %d",
      pos.x, pos.y, eta, feedback->number_of_recoveries);
  }

  //  Callback 3: Ket qua cuoi cung
  void ResultCallback(const GoalHandleNav::WrappedResult & result)
  {
    switch (result.code) {
      case rclcpp_action::ResultCode::SUCCEEDED:
        RCLCPP_INFO(get_logger(), ">>> SUCCEEDED: Robot da den dich thanh cong! <<<")
        break;

      case rclcpp_action::ResultCode::ABORTED:
        RCLCPP_ERROR(get_logger(), ">>> ABORTED: Robot khong the den dich (bi ket hoa
        break;

      case rclcpp_action::ResultCode::CANCELED:
        RCLCPP_WARN(get_logger(), ">>> CANCELED: Goal da bi huy.");
        break;

      default:
        RCLCPP_ERROR(get_logger(), ">>> UNKNOWN result code.");
        break;
    }
    rclcpp::shutdown();
  }
};
```

```
int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<GoalSenderNode>();

  // Gui goal den toa do (1.5, -0.5), huong thang (yaw = 0)
  node->send_goal(1.5, -0.5, 0.0);

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```

6.4 Execution

The node was built and executed using the following commands:

```
cd ~/ros2_ws
colcon build --packages-select nav2_goal_sender
source install/setup.bash
ros2 run nav2_goal_sender goal_sender
```

6.5 Results

- The robot successfully received the navigation goal.
- The robot moved towards the target position in the map.
- Feedback messages were received during navigation, including current pose and estimated time remaining.
- The result callback indicated successful completion of the goal.

lab4_result.png

Figure 6: Robot navigating to the target goal in RViz

6.6 Analysis

- The action client allows asynchronous communication with the navigation stack, enabling real-time monitoring of the robot's progress.
- The feedback callback provides useful information such as current position and estimated time, which can be used for debugging or visualization.
- Proper conversion from yaw to quaternion is essential to ensure correct orientation of the goal.
- The use of callbacks (goal response, feedback, result) makes the system flexible and responsive.

6.7 Discussion

1. Why is an action used instead of a service?

Actions are used because navigation is a long-running task. Unlike services, actions allow feedback during execution and support cancellation.

2. What is the role of feedback in the action?

Feedback provides real-time updates about the robot's progress, such as current pose and estimated remaining time. This helps monitor the execution and improves system transparency.

7 Lab 5 – Planner and Controller Configuration

7.1 Objective

The objective of this lab is to understand how different planners and controllers in NAV2 affect robot navigation. Specifically, we compare the default configuration (NavFn + DWB) with an alternative configuration (SmacPlanner2D + Regulated Pure Pursuit).

7.2 Configuration

The NAV2 parameters file was modified to change both the global planner and the local controller.

Global Planner (SmacPlanner2D):

```
planner_server:
ros__parameters:
planner_plugins: ["GridBased"]
GridBased:
plugin: "nav2_smac_planner/SmacPlanner2D"
tolerance: 0.5
use_astar: true
allow_unknown: true
```

Local Controller (Regulated Pure Pursuit):

```
controller_server:
ros__parameters:
controller_plugins: ["FollowPath"]
FollowPath:
plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
desired_linear_vel: 0.2
lookahead_dist: 0.6
min_lookahead_dist: 0.3
max_lookahead_dist: 0.9
```

7.3 Execution

NAV2 was launched using the custom parameter file:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py
use_sim_time:=True
map:=$HOME/maps/turtlebot3_world.yaml
params_file:=$HOME/my_nav2_params.yaml
```

Navigation goals were sent using RViz2 for both configurations:

- Default: NavFn + DWB
- Modified: SmacPlanner2D + Regulated Pure Pursuit

7.4 Results

Case 1: NavFn + DWB

- Path shape: The path is generally smooth but not always optimal in length.
- Robot behavior: Slightly jerky due to velocity sampling.
- Obstacle avoidance: Average

lab5_navfn.png

Figure 7: Navigation using NavFn + DWB

Case 2: SmacPlanner2D + Regulated Pure Pursuit

- Path shape: The path is more optimal and efficient, with better alignment to free space.
- Robot behavior: Smooth and stable motion.
- Obstacle avoidance: Better compared to NavFn + DWB

lab5_smac.png

Figure 8: Navigation using SmacPlanner2D + Regulated Pure Pursuit

7.5 Analysis

- The NavFn planner typically generates smooth paths but does not always produce optimal trajectories.
- The SmacPlanner2D uses A* and can generate more efficient and feasible paths, especially in complex environments.
- The DWB controller may produce jerky motion due to velocity sampling and cost evaluation.
- The Regulated Pure Pursuit controller results in smoother motion and better path tracking for differential drive robots like TurtleBot3.
- The combination of SmacPlanner2D and Regulated Pure Pursuit generally improves navigation performance in terms of smoothness and stability.

7.6 Discussion

1. What is the difference between NavFn and SmacPlanner2D?

NavFn is based on Dijkstra's algorithm, which guarantees finding a path but may not be optimal. SmacPlanner2D uses A* search, which is more efficient and can produce shorter paths by incorporating heuristics.

2. Why is Regulated Pure Pursuit suitable for TurtleBot3?

Regulated Pure Pursuit is well-suited for differential drive robots because it generates smooth trajectories and avoids abrupt velocity changes. It is computationally efficient and provides stable control.

8 Lab 6 – Behavior Tree in NAV2

8.1 Objective

The objective of this lab is to understand how Behavior Trees (BT) are used in NAV2 to control the navigation process, and to modify the default BT to observe changes in robot behavior.

8.2 Overview

NAV2 uses Behavior Trees to coordinate planning, control, and recovery behaviors. A Behavior Tree consists of different types of nodes such as:

- **Sequence**: Executes children in order until one fails
- **Fallback**: Executes children until one succeeds
- **Action**: Performs a specific task
- **Condition**: Checks a condition

8.3 Default Behavior Tree

The default BT file was located using:

```
find /opt/ros/humble -name "navigate_to_pose_w_replanning_and_recovery.xml"
```

The file was opened using Groot2 for visualization.

8.4 Analysis of Behavior Tree

1. Position of ComputePathToPose

The node `ComputePathToPose` is located in the planning section of the Behavior Tree. It is responsible for generating a global path from the current position to the goal.

2. Role of RoundRobin

The `RoundRobin` node is used to cycle through recovery behaviors when navigation fails. It ensures that different recovery strategies are attempted sequentially.

3. Sequence and Fallback

- **Sequence**: Used when all steps must succeed (e.g., compute path → follow path)
- **Fallback**: Used when alternative actions are needed if one fails (e.g., recovery behaviors)

8.5 Modification

The Behavior Tree was modified by adding a wait action before the spin recovery:

```
<Fallback> <Sequence> <IsStuck/> <Wait wait_duration="2.0"/> <Spin spin_dist="1.57"/>
```

8.6 Results

- The robot entered recovery behavior when it was stuck.
- A delay of approximately 2 seconds was observed before the robot started spinning.
- The modified Behavior Tree successfully altered the recovery sequence.

lab6_bt.png

Figure 9: Behavior Tree visualization in Groot2

8.7 Analysis

- Behavior Trees provide a flexible and modular way to control robot behavior.
- The addition of the Wait node introduces a delay, allowing the system to stabilize before executing recovery actions.
- The recovery mechanism ensures robustness in navigation by handling failure cases.

8.8 Discussion

1. Why does NAV2 use Behavior Trees?

Behavior Trees allow modular and hierarchical control of complex behaviors. They are easier to extend and debug compared to traditional state machines.

2. What is the benefit of adding a Wait node?

Adding a Wait node introduces a delay before executing recovery actions, which can prevent unnecessary or aggressive recovery behaviors.

9 Homework 1 – Patrol Robot

9.1 Objective

The objective of this assignment is to implement a ROS2 C++ node that enables the TurtleBot3 robot to autonomously patrol through a set of predefined waypoints in a loop.

9.2 Requirements

The robot must:

- Navigate through 4 predefined waypoints sequentially
- Pause for 3 seconds at each waypoint
- Repeat the patrol loop indefinitely
- Handle navigation failures (ABORTED state)
- Log status messages during execution

9.3 Implementation

A ROS2 node was implemented using `rclcpp_action` to send navigation goals to the `/navigate_to_pose` action server.

Key features:

- A list of waypoints stored as (x, y, yaw)
- Sequential goal sending using action client
- Timer or delay mechanism for waiting at each waypoint
- Result callback to trigger next goal
- Handling of ABORTED goals by skipping to the next waypoint

9.4 Code

Main implementation:

```
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_action/rclcpp_action.hpp>
#include <nav2_msgs/action/navigate_to_pose.hpp>
#include <geometry_msgs/msg/pose_stamped.hpp>
#include <cmath>
#include <tuple>
#include <vector>

using NavigateToPose = nav2_msgs::action::NavigateToPose;
using GoalHandleNav  = rclcpp_action::ClientGoalHandle<NavigateToPose>;

class PatrolNode : public rclcpp::Node
{
public:
  PatrolNode() : Node("patrol_node"), m_current_index(0), m_is_running(true)
  {
    m_nav_client = rclcpp_action::create_client<NavigateToPose>(this, "navigate_to_pos

    // 4 waypoint tun tra trong turtlebot3_world: {x, y, yaw_rad}
    m_waypoints = {
      { 1.5, -0.5,  0.0},
      { 1.5,  1.5,  1.5708},   // ~90°
      {-1.0,  1.5,  3.1416},   // ~180°
      {-1.0, -0.5, -1.5708},   // ~-90°
    };

    // Graceful shutdown khi nhn Ctrl+C
    rclcpp::on_shutdown([this]() {
      m_is_running = false;
      RCLCPP_INFO(this->get_logger(), "Patrol node shutting down gracefully.");
    });

    RCLCPP_INFO(this->get_logger(),
      "PatrolNode started. %zu waypoints loaded. Waiting for action server...",
      m_waypoints.size());

    // Ch action server sn sàng trc khi bt tun tra
    if (!m_nav_client->wait_for_action_server(std::chrono::seconds(10))) {
      RCLCPP_ERROR(this->get_logger(), "Action server /navigate_to_pose not available
      rclcpp::shutdown();
      return;
    }

    RCLCPP_INFO(this->get_logger(), "Action server ready. Starting patrol...");
    SendNextGoal();
```

```
  }

private:
  //  Gi goal đn waypoint tip theo trong danh sách
  void SendNextGoal()
  {
    if (!m_is_running || !rclcpp::ok()) return;

    auto [x, y, yaw] = m_waypoints[m_current_index];

    RCLCPP_INFO(this->get_logger(),
      " Navigating to waypoint [%zu/%zu]: (%.2f, %.2f, yaw=%.2f)",
      m_current_index + 1, m_waypoints.size(), x, y, yaw);

    auto goal_msg = NavigateToPose::Goal();
    goal_msg.pose.header.frame_id = "map";
    goal_msg.pose.header.stamp    = this->now();
    goal_msg.pose.pose.position.x  = x;
    goal_msg.pose.pose.position.y  = y;
    goal_msg.pose.pose.position.z  = 0.0;

    // Chuyn yaw → quaternion (ch xoay quanh trc z)
    goal_msg.pose.pose.orientation.z = std::sin(yaw / 2.0);
    goal_msg.pose.pose.orientation.w = std::cos(yaw / 2.0);

    auto send_goal_options = rclcpp_action::Client<NavigateToPose>::SendGoalOptions()

    send_goal_options.goal_response_callback =
      [this](const GoalHandleNav::SharedPtr & goal_handle)
      { GoalResponseCallback(goal_handle); };

    send_goal_options.feedback_callback =
      [this](GoalHandleNav::SharedPtr,
             const std::shared_ptr<const NavigateToPose::Feedback> feedback)
      { FeedbackCallback(feedback); };

    send_goal_options.result_callback =
      [this](const GoalHandleNav::WrappedResult & result)
      { ResultCallback(result); };

    m_nav_client->async_send_goal(goal_msg, send_goal_options);
  }

  //  Goal chp nhn / b t chi
  void GoalResponseCallback(const GoalHandleNav::SharedPtr & goal_handle)
  {
    if (!goal_handle) {
      RCLCPP_ERROR(this->get_logger(),
```

```
      "Goal to waypoint [%zu] was REJECTED by action server!", m_current_index + 1)
    AdvanceAndSend();
  } else {
    RCLCPP_INFO(this->get_logger(),
      "Goal to waypoint [%zu] ACCEPTED.", m_current_index + 1);
  }
}

//  Feedback: in v trí hin ti và ETA
void FeedbackCallback(
  const std::shared_ptr<const NavigateToPose::Feedback> feedback)
{
  auto & pos = feedback->current_pose.pose.position;
  double eta = rclcpp::Duration(feedback->estimated_time_remaining).seconds();
  RCLCPP_INFO(this->get_logger(),
    "   Current pos: (%.2f, %.2f) | ETA: %.1f s",
    pos.x, pos.y, eta);
}

//  Kt qu: SUCCEEDED / ABORTED / CANCELED
void ResultCallback(const GoalHandleNav::WrappedResult & result)
{
  switch (result.code) {
    case rclcpp_action::ResultCode::SUCCEEDED:
      RCLCPP_INFO(this->get_logger(),
        " Arrived at waypoint [%zu]! Waiting 3 seconds...",
        m_current_index + 1);
      // Dng 3 giây ti waypoint ri mi chuyn đim tip
      rclcpp::sleep_for(std::chrono::seconds(3));
      AdvanceAndSend();
      break;

    case rclcpp_action::ResultCode::ABORTED:
      RCLCPP_WARN(this->get_logger(),
        " Goal ABORTED at waypoint [%zu]. Skipping to next waypoint.",
        m_current_index + 1);
      AdvanceAndSend();
      break;

    case rclcpp_action::ResultCode::CANCELED:
      RCLCPP_WARN(this->get_logger(),
        " Goal CANCELED at waypoint [%zu]. Skipping to next waypoint.",
        m_current_index + 1);
      AdvanceAndSend();
      break;

    default:
      RCLCPP_ERROR(this->get_logger(), "Unknown result code. Skipping.");
```

```
        AdvanceAndSend();
        break;
    }
  }

  //  Tăng index theo vòng (wrap-around) và gi goal tip theo
  void AdvanceAndSend()
  {
    if (!m_is_running || !rclcpp::ok()) return;

    m_current_index = (m_current_index + 1) % m_waypoints.size();

    // Nu va quay v index 0 → bt đu vòng mi
    if (m_current_index == 0) {
      RCLCPP_INFO(this->get_logger(),
        " Completed one full patrol loop! Starting again... ");
    }

    SendNextGoal();
  }

  rclcpp_action::Client<NavigateToPose>::SharedPtr m_nav_client;
  std::vector<std::tuple<double, double, double>>  m_waypoints;
  size_t m_current_index;
  bool   m_is_running;
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<PatrolNode>());
  rclcpp::shutdown();
  return 0;
}
```

9.5 Execution

The node was built and executed using:

```
cd ~/ros2_ws
colcon build --packages-select nav2_goal_sender
source install/setup.bash
ros2 run nav2_goal_sender patrol_node
```

9.6 Results

- The robot successfully navigated through all predefined waypoints.
- The robot paused at each waypoint for approximately 3 seconds.

- The patrol loop repeated continuously.
- When a goal was aborted, the robot skipped to the next waypoint.

hw1_patrol.png

Figure 10: Robot patrolling through multiple waypoints in RViz

9.7 Analysis

- The use of an action client allows asynchronous navigation and continuous operation.
- The waypoint list structure simplifies patrol logic and makes it easy to extend.
- Handling the ABORTED state improves system robustness by preventing the robot from getting stuck.
- Introducing a delay between goals ensures stable transitions and avoids abrupt behavior.

9.8 Discussion

1. Why is a loop required for patrol behavior?

A loop ensures continuous operation, allowing the robot to repeatedly monitor or cover an area without manual intervention.

2. How does the system handle navigation failure?

When a goal is aborted, the system logs a warning and proceeds to the next waypoint, ensuring that the patrol process continues.

10 Homework 2 – Planner Comparison using Metrics

10.1 Objective

The objective of this assignment is to compare the performance of two global planners in NAV2, namely NavFn and SmacPlanner2D, using quantitative metrics such as execution time and traveled distance.

10.2 Methodology

A ROS2 C++ node was implemented to automatically send navigation goals and collect performance data.

Evaluation metrics:

- Execution time: measured from goal submission to successful completion
- Travel distance: computed from accumulated odometry data
- Recovery behavior: whether recovery actions were triggered

Each planner was tested multiple times using the same start and goal positions to ensure consistency.

10.3 Implementation

The node performs the following steps:

- Send a navigation goal using `/navigate`$_t o_p ose$
- `Record the start time`
- `Monitor feedback and result`
- `Calculate execution time`
- `Subscribe to /odom and accumulate traveled distance`

Code:

```
// Paste your evaluation_node.cpp here
```

10.4 Data Collection

Data was recorded using ROS2 bag:

```
ros2 bag record /odom /plan /navigate_to_pose/_action/feedback
```

Each planner was tested 5 times.

10.5 Results

Due to simulation limitations, quantitative data could not be collected. The comparison is based on theoretical analysis.

10.6 Analysis

- NavFn generally produces feasible paths but may not be optimal in terms of distance or time.
- SmacPlanner2D, which uses A* search, tends to generate shorter and more efficient paths.
- In some cases, NavFn may trigger recovery behaviors due to less optimal planning.
- SmacPlanner2D often results in smoother navigation with fewer recovery actions.
- The difference in performance becomes more significant in complex environments.

10.7 Discussion

1. Which planner performs better?

Based on the collected metrics, SmacPlanner2D generally outperforms NavFn in terms of efficiency and path optimality.

2. Why is SmacPlanner2D more efficient?

SmacPlanner2D uses the A* algorithm, which incorporates heuristics to guide the search process, resulting in faster and shorter paths compared to Dijkstra-based methods.

11 Homework 3 – Custom NAV2 Controller Plugin

11.1 Objective

The objective of this assignment is to implement a custom controller plugin for NAV2 using C++, based on the `nav2_core::Controller` interface. The controller follows a Pure Pursuit strategy to guide the robot along a given path.

11.2 Overview

The Pure Pursuit algorithm is a geometric path tracking method. It selects a look-ahead point on the global path and computes the angular velocity required to steer the robot toward that point.

11.3 Implementation Details

The custom controller `PurePursuitController` was implemented with the following logic:

- The global path is received via `setPlan()` and stored internally.
- A look-ahead point is selected based on a fixed distance parameter `lookahead_dist`.

- The heading error α is computed between the robot's orientation and the direction to the look-ahead point.
- The angular velocity is calculated as: $[\ \omega = k \cdot \alpha\ ]$ where k is the parameter `angular_gain`.
- The linear velocity is set to a constant value `linear_vel`.
- The angular velocity is limited by `max_angular_vel` to ensure stability.
- When the robot is within `goal_tolerance` of the final waypoint, the controller outputs zero velocity.

11.4 Parameters

The controller uses the following parameters:

- `lookahead_dist` = 0.5 m
- `linear_vel` = 0.15 m/s
- `angular_gain` = 1.5
- `goal_tolerance` = 0.25 m
- `max_angular_vel` = 1.0 rad/s

11.5 Plugin Registration

The plugin was registered using `pluginlib`:

```
PLUGINLIB_EXPORT_CLASS(
my_pure_pursuit::PurePursuitController,
nav2_core::Controller)
```

11.6 Integration with NAV2

The plugin was integrated into NAV2 through the parameter file:

```
controller_server:
ros__parameters:
controller_plugins: ["FollowPath"]
FollowPath:
plugin: "my_pure_pursuit/PurePursuitController"
```

11.7 Results

- The custom controller was successfully loaded without runtime errors.
- The robot followed the planned path using the Pure Pursuit algorithm.
- The motion of the robot was smooth with slight oscillations when turning sharply.
- The robot reached the goal within the defined tolerance.

hw3_result.png

Figure 11: Robot following path using custom Pure Pursuit controller

11.8 Analysis

- The controller provides a simple yet effective path tracking mechanism.
- The parameter `lookahead_dist` significantly affects behavior:
  - Smaller values increase tracking accuracy but may cause oscillation
  - Larger values improve smoothness but reduce precision
- The use of `angular_gain` allows tuning of turning responsiveness.
- Limiting angular velocity using `max_angular_vel` prevents unstable rotation.
- The controller works well for differential drive robots such as TurtleBot3.

11.9 Discussion

1. How is angular velocity computed?

Angular velocity is computed from the heading error between the robot and the look-ahead point, scaled by the parameter `angular_gain`.

2. Why is look-ahead distance important?

The look-ahead distance determines how far ahead the robot tracks the path. It directly controls the trade-off between smoothness and accuracy.